

Practical 1 — Data Loading, Cleaning & Correlation

```
import pandas as pd

import numpy as np

import matplotlib.pyplot as plt

import seaborn as sns

# Using Seaborn's built-in 'tips' dataset (or load your own CSV)

df = sns.load_dataset('tips')

print('Shape:', df.shape)

print(df.head())
```

```
Shape: (244, 7)

   total_bill  tip  sex smoker  day  time  size
0      16.99  1.01 Female    No  Sun  Dinner    2
```

Identify & Count Missing Values

```
# Count missing values per column

print('Missing values per column:')

print(df.isnull().sum())

# Total missing

print('Total missing:', df.isnull().sum().sum())
```

1a — Drop Duplicate Rows

```
print('Rows before:', len(df))

df = df.drop_duplicates()

print('Rows after dropping duplicates:', len(df))
```

1b — Detect & Remove Outliers (IQR Method)

```
def remove_outliers_iqr(dataframe, column):

    Q1 = dataframe[column].quantile(0.25)

    Q3 = dataframe[column].quantile(0.75)
```

```

IQR = Q3 - Q1

lower =Q1 - 1.5 * IQR

upper =Q3 + 1.5 * IQR

return dataframe[(dataframe[column] >= lower) &
                  (dataframe[column] <= upper)]

# Apply to numeric columns

numeric_cols = df.select_dtypes(include=np.number).columns.tolist()

print('Numeric columns:', numeric_cols)

print('Rows before outlier removal:', len(df))

for col in numeric_cols:

    df = remove_outliers_iqr(df, col)

print('Rows after outlier removal:', len(df))

```

1c — Most Positively & Negatively Correlated Attributes

```

# Compute correlation matrix on numeric columns

corr_matrix = df[numeric_cols].corr()

# Unstack and sort (drop self-correlations)

corr_pairs = (corr_matrix
              .where(np.triu(np.ones(corr_matrix.shape), k=1).astype(bool))
              .stack()
              .reset_index())

corr_pairs.columns = ['Feature1', 'Feature2', 'Correlation']

corr_pairs = corr_pairs.sort_values('Correlation', ascending=False)

print('Most Positively Correlated:')

print(corr_pairs.head(3).to_string(index=False))

print('Most Negatively Correlated:')

print(corr_pairs.tail(3).to_string(index=False))

# Heatmap

plt.figure(figsize=(7, 5))

sns.heatmap(corr_matrix, annot=True, fmt='.2f', cmap='coolwarm')

plt.title('Correlation Heatmap')

```

Practical 2 — Iris Dataset: Descriptive Statistics & Correlation

Setup — Load Iris Dataset

```
import pandas as pd

import numpy as np

import matplotlib.pyplot as plt

import seaborn as sns

from scipy import stats

from sklearn.datasets import load_iris

iris = load_iris()

df = pd.DataFrame(iris.data, columns=iris.feature_names)

df['species'] = pd.Categorical.from_codes(iris.target, iris.target_names)

print(df.head())

print('Shape:', df.shape)
```

2i — Mean, Mode, Median, SD, Confidence Interval & Standard Error

```
features = iris.feature_names

for feature in features:

    data = df[feature]

    mean = data.mean()

    mode = data.mode()[0]

    median = data.median()

    std = data.std()

    se = stats.sem(data)                # Standard Error

    ci = stats.t.interval(0.95, df=len(data)-1, loc=mean, scale=se)

    print(f'--- {feature} ---')

    print(f'  Mean    : {mean:.4f}')

    print(f'  Mode     : {mode:.4f}')

    print(f'  Median   : {median:.4f}')

    print(f'  Std Dev: {std:.4f}')
```

```
print(f' Std Err: {se:.4f}')
```

```
print(f' 95% CI : ({ci[0]:.4f}, {ci[1]:.4f}))')
```

```
print()
```

```
--- sepal length (cm) ---
```

```
Mean    : 5.8433
```

```
Mode     : 5.0000
```

```
Median  : 5.8000
```

```
Std Dev: 0.8281
```

```
Std Err: 0.0657
```

```
95% CI  : (5.7133, 5.9733)
```

```
(... repeated for each feature ...)
```

2ii — Correlation Coefficients & Heatmap

```
# Pearson correlation matrix
```

```
corr = df[features].corr()
```

```
print('Correlation Matrix:')
```

```
print(corr.round(3))
```



```
# Plot heatmap
```

```
plt.figure(figsize=(8, 6))
```

```
sns.heatmap(corr, annot=True, fmt='.3f', cmap='RdYlGn',
```

```
            linewidths=0.5, linecolor='white',
```

```
            vmin=-1, vmax=1)
```

```
plt.title('Iris Feature Correlation Heatmap', fontsize=14)
```

```
plt.tight_layout()
```

```
plt.show()
```

2iii — Covariance Between Sepal Length and Petal Length

```
cov_matrix = df[features].cov()
```



```
sepal_len = 'sepal length (cm)'
```

```
petal_len = 'petal length (cm)'
```



```
cov_val = cov_matrix.loc[sepal_len, petal_len]
```

```
print(f'Covariance between sepal length and petal length: {cov_val:.4f}')
```

```
# Full covariance matrix

print('Full Covariance Matrix:')

print(cov_matrix.round(3))
```

Covariance between sepal length and petal length: 1.2743

2iv — Contingency Table for Class (Species) Feature

```
# Simple frequency count

print('Class Frequency:')

print(df['species'].value_counts())

# Cross-tabulation with a binned feature

df['sepal_bin'] = pd.cut(df['sepal length (cm)'],

                        bins=3,

                        labels=['Small', 'Medium', 'Large'])

contingency = pd.crosstab(df['sepal_bin'], df['species'])

print('Contingency Table (Sepal Length Bin vs Species):')

print(contingency)
```

species	setosa	versicolor	virginica
sepal_bin			
Small	39	3	0
Medium	11	42	20
Large	0	5	30

Practical 3 — Titanic Dataset: Visualization

Setup — Load Titanic Dataset

```
import pandas as pd

import numpy as np

import matplotlib.pyplot as plt

import seaborn as sns

# Load via seaborn (easy method)

titanic = sns.load_dataset('titanic')

# Drop rows with critical missing values

titanic.dropna(subset=['age', 'fare', 'sex', 'survived'], inplace=True)

print('Shape:', titanic.shape)

print(titanic[['survived', 'sex', 'age', 'fare', 'pclass']].head())
```

3a — Bar Chart: Survivors vs Non-Survivors by Gender

```
# Group data

survival_gender = titanic.groupby(['sex', 'survived']).size().unstack()

survival_gender.columns = ['Did Not Survive', 'Survived']

fig, ax = plt.subplots(figsize=(8, 5))

survival_gender.plot(kind='bar', ax=ax,

                    color=['#ef5350', '#42a5f5'],

                    edgecolor='black', width=0.6)

ax.set_title('Survivor Frequency by Gender', fontsize=14, fontweight='bold')

ax.set_xlabel('Gender', fontsize=12)

ax.set_ylabel('Count of Passengers', fontsize=12)

ax.set_xticklabels(ax.get_xticklabels(), rotation=0)

ax.legend(title='Outcome')

plt.tight_layout()

plt.show()
```

3b — Scatter Plot: Age vs Fare

```

fig, ax = plt.subplots(figsize=(8, 5))

colors_map = {0: '#ef5350', 1: '#66bb6a'}

for outcome, group in titanic.groupby('survived'):

    label = 'Survived' if outcome == 1 else 'Did Not Survive'

    ax.scatter(group['age'], group['fare'],

               c=colors_map[outcome], label=label,

               alpha=0.6, edgecolors='w', linewidths=0.5, s=50)

ax.set_title('Age vs Fare (coloured by Survival)', fontsize=14)
ax.set_xlabel('Age', fontsize=12)
ax.set_ylabel('Fare (USD)', fontsize=12)
ax.legend(title='Outcome')

plt.tight_layout()

plt.show()

```

3c — Density Distribution: Age & Passenger Fare

```

fig, axes = plt.subplots(1, 2, figsize=(12, 5))

# Age distribution
sns.kdeplot(data=titanic, x='age', hue='survived',

            fill=True, ax=axes[0], palette=['#ef5350', '#42a5f5'])
axes[0].set_title('Density Distribution - Age')
axes[0].set_xlabel('Age')
axes[0].set_ylabel('Density')

# Fare distribution
sns.kdeplot(data=titanic, x='fare', hue='survived',

            fill=True, ax=axes[1], palette=['#ef5350', '#42a5f5'])
axes[1].set_title('Density Distribution - Passenger Fare')
axes[1].set_xlabel('Fare (USD)')
axes[1].set_ylabel('Density')

plt.suptitle('Density Comparison by Survival', fontsize=13, fontweight='bold')

plt.tight_layout()

plt.show()

```

3d — Pair Plot: Pairwise Bivariate Distribution

```
pair_features = ['survived', 'age', 'fare', 'pclass']

pair_df = titanic[pair_features].copy()
pair_df['survived'] = pair_df['survived'].map({0: 'No', 1: 'Yes'})

g = sns.pairplot(pair_df, hue='survived',
                  palette={'Yes': '#42a5f5', 'No': '#ef5350'},
                  plot_kws={'alpha': 0.5},
                  diag_kind='kde')

g.figure.suptitle('Pairwise Bivariate Distribution - Titanic',
                  y=1.02, fontsize=13)

plt.show()
```

Practical 4 — Titanic Dataset: Data Analysis

Setup (reuse Titanic df from Practical 3)

```
import pandas as pd

import seaborn as sns

titanic = sns.load_dataset('titanic')

# Keep all rows for analysis (do not drop NaN in age for Q4a)
```

4a — Total Passengers with Age < 30

```
count_under_30 = titanic[titanic['age'] < 30]['age'].count()

print(f'Passengers with age < 30: {count_under_30}')

# Percentage

total_with_age = titanic['age'].dropna().count()

pct = (count_under_30 / total_with_age) * 100

print(f'Percentage: {pct:.1f}%')
```

```
Passengers with age < 30: 369

Percentage: 51.5%
```

4b — Total Fare Paid by First-Class Passengers

```
first_class = titanic[titanic['pclass'] == 1]

total_fare = first_class['fare'].sum()

avg_fare = first_class['fare'].mean()

count_fc = len(first_class)

print(f'First-class passengers : {count_fc}')

print(f'Total fare collected : ${total_fare:.2f}')

print(f'Average fare per person: ${avg_fare:.2f}')
```

```
First-class passengers : 216

Total fare collected : $18177.41

Average fare per person: $84.15
```

4c — Survivors per Passenger Class

```

survivor_class = titanic.groupby('pclass')['survived'].agg(['sum', 'count'])

survivor_class.columns = ['Survivors', 'Total']

survivor_class['Survival Rate (%)'] = (
    (survivor_class['Survivors'] / survivor_class['Total']) * 100
).round(1)

print(survivor_class)

# Bar chart comparison
import matplotlib.pyplot as plt

fig, ax = plt.subplots(figsize=(7, 5))

x = survivor_class.index

ax.bar(x - 0.2, survivor_class['Total'], 0.4, label='Total', color='#90caf9')

ax.bar(x + 0.2, survivor_class['Survivors'], 0.4, label='Survived', color='#66bb6a')

ax.set_title('Survivors per Passenger Class', fontsize=13, fontweight='bold')
ax.set_xlabel('Passenger Class')
ax.set_ylabel('Count')
ax.set_xticks(x)
ax.set_xticklabels(['1st Class', '2nd Class', '3rd Class'])
ax.legend()

plt.tight_layout()

plt.show()

```

	Survivors	Total	Survival Rate (%)
pclass			
1	136	216	63.0
2	87	184	47.3
3	119	491	24.2

Practical 5 — Custom Dataset: Exploratory Analysis

Setup — Load Dataset

```
import pandas as pd

import numpy as np

import matplotlib.pyplot as plt

import seaborn as sns

df = sns.load_dataset('diamonds')

print('Shape:', df.shape)

print(df.dtypes)

print(df.head(3))
```

5a — Count Categorical & Numeric Features

```
numeric_cols = df.select_dtypes(include=np.number).columns.tolist()

categorical_cols = df.select_dtypes(include=['object', 'category']).columns.tolist()

print(f'Numeric features ({len(numeric_cols)}): {numeric_cols}')

print(f'Categorical features ({len(categorical_cols)}): {categorical_cols}')
```

```
Numeric features (7): ['carat', 'depth', 'table', 'price', 'x', 'y', 'z']

Categorical features (3): ['cut', 'color', 'clarity']
```

5b — Remove One Correlated Attribute

```
# Compute correlation matrix for numeric features

corr = df[numeric_cols].corr().abs() # absolute values

# Get upper triangle pairs

upper = corr.where(np.triu(np.ones(corr.shape), k=1).astype(bool))

# Find columns with correlation > 0.90

to_drop = [col for col in upper.columns if any(upper[col] > 0.90)]

print('Highly correlated columns to drop:', to_drop)

df_cleaned = df.drop(columns=to_drop)
```

```
print('Columns after removal:', df_cleaned.columns.tolist())
```

```
Highly correlated columns to drop: ['y', 'z']
```

```
Columns after removal: ['carat', 'cut', 'color', 'clarity', 'depth', 'table', 'price', 'x']
```

5c — Five-Number Summary of Each Attribute (Visual)

```
remaining_numeric = df_cleaned.select_dtypes(include=np.number).columns.tolist()

# Textual five-number summary
summary = df_cleaned[remaining_numeric].describe()
five_num = summary.loc[['min', '25%', '50%', '75%', 'max']]
print('Five-Number Summary:')
print(five_num.round(2))

# Visual - Box plots for all numeric features
fig, axes = plt.subplots(2, 3, figsize=(14, 8))
axes = axes.flatten()

for i, col in enumerate(remaining_numeric):
    axes[i].boxplot(df_cleaned[col].dropna(),
                    patch_artist=True,
                    boxprops=dict(facecolor='#bbdefb', color='#1565c0'),
                    medianprops=dict(color='#e53935', linewidth=2),
                    whiskerprops=dict(color='#1565c0'),
                    capprops=dict(color='#1565c0'),
                    flierprops=dict(marker='o', color='gray',
                                     alpha=0.4, markersize=3))

    axes[i].set_title(col, fontsize=11, fontweight='bold')
    axes[i].set_ylabel('Value')
    axes[i].grid(axis='y', linestyle='--', alpha=0.5)

# Hide unused subplots
for j in range(len(remaining_numeric), len(axes)):
    axes[j].set_visible(False)

plt.suptitle('Five-Number Summary - Box Plots (Diamonds Dataset)',
             fontsize=13, fontweight='bold')
```